Evaluating the Multi-Agent Coordination Tax: Dynamic Routing Across Architectural Topologies

1st Yian Chen
Department of Electrical and Computer Engineering
Stevens Institute of Technology
Hoboken, USA
ychen17@stevens.edu

2nd Hong Man
Department of Electrical and Computer Engineering
Stevens Institute of Technology
Hoboken, USA
hman@stevens.edu

***Abstract*—As agentic systems scale, the prevailing industry assumption that "more agents are inherently better" becomes computationally and economically flawed. Building upon empirical scaling laws, this paper introduces a Dynamic Architecture Router designed to predictively dispatch tasks to either a Single-Agent System (SAS) or a Centralized Multi-Agent System (CMAS). To evaluate this, we introduce "Routing Regret" — a composite metric of trajectory accuracy, inference latency, and token efficiency. Through evaluations on customized Personalized Context Assembly Benchmark (PCAB), we demonstrate that while MAS architectures often yield higher accuracy, they suffer from a serious structural "Coordination Tax." Initial CMAS runs resulted in catastrophic context overflows, leading to severe penalties in our composite reward evaluation. Even after the addition of multi-layer telemetry and circuit-breaking mechanisms, the stabilized CMAS still shows unavoidable structural tax due to exhaustive communication patterns. Using a mathematically weighted Composite Reward function, the Oracle baseline selected the SAS topology in 100% of our test cases, validating the strict necessity of predictive architectural routing for enterprise generative AI.[1]**

***Index Terms*—Multi-Agent Systems, Large Language Models, Orchestration, Dynamic Routing, Agentic Workflows**

I. Introduction

A. Background and Importance

The transition from single-turn, reactive large language models (LLMs) to autonomous, multi-step agentic systems represents a fundamental paradigm shift in artificial intelligence. LLM-based agentic systems are capable of reasoning, planning, and external environment interaction through dynamic tool use, and are in turn increasingly positioned as the primary mechanism for automating complex workflows.

As enterprise adoption of generative AI accelerates, inference cost and execution latency have become paramount concerns [1]. The economic and operational viability of deploying agentic systems in production depends on compute-optimal test-time scaling [2]. If an automated workflow consumes excessive API tokens or requires too much wall-clock time to execute a simple sequential task, its utility is severely diminished. Consequently, achieving architectural efficiency in agentic orchestration becomes critical for scalable enterprise deployment.

B. Challenges in Agentic Orchestration

Orchestrating these systems presents significant technical challenges. The prevailing design heuristic often relies on trial-and-error or the assumption that "more agents are always better," leading developers to construct multi-agent swarms under the belief that collective reasoning is superior to individual capability [3]. However, such scaling hypothesis proves to be both computationally and economically unsustainable.

Recent technical literature has also challenged this assumption. Researchers at Google DeepMind and MIT conducted an exhaustive evaluation of 180 agent configurations, revealing that while multi-agent coordination can improve performance on highly parallelizable tasks, it significantly degrades performance on tasks that require strict sequential constraint satisfaction, with performance drops ranging from 39% to 70% [4]. Li [5] corroborates this, emphasizing that capable Single-Agent Systems (SAS) often outperform complex MAS when the administrative overhead of inter-agent communication outweighs the cognitive benefits.

A primary challenge lies in what is known as the "Multi-Agent Coordination Tax". In a Centralized MAS, bidirectional state synchronization causes token consumption to scale quadratically ($O(N^2)$), often leading to silent inefficiencies or catastrophic context window overflows. Furthermore, failure diagnostics in multi-agent orchestration remain rudimentary, making it difficult to differentiate between a generic LLM hallucination, structural architectural problem, or a system design issue.

C. Related Work

This architectural shift and the need for intelligent orchestration are reflected across the current engineering landscape. Recent surveys emphasize the critical need to minimize latency [6] and formalize agentic software engineering [7]. Multi-Agent Systems are increasingly deployed in specialized industrial applications, ranging from financial frameworks [8] and electronic design automation [9], to digital twin simulations [10], self-organizing networks [11], computer-automated design [12], and human-machine robotics [13].

To address the lack of diagnostic visibility in these deployments, researchers at UC Berkeley recently introduced the Multi-Agent System Failure Taxonomy (MAST), identifying

[1]Code available at https://github.com/yianan261/Dynamic-Architecture-Router-for-Agentic-Systems

distinct breakdown categories such as Inter-Agent Misalignment and System Design Exhaustion [14].

D. Current Contribution and Progress

Building upon these foundational challenges, this research moves beyond static topologies by treating system architecture as a dynamic variable. The main contributions of this paper are:

1) We propose a predictive **Dynamic Architecture Router**, a pre-execution meta-agent that extracts heuristic features from user queries to optimally route tasks to either a Single-Agent or Centralized Multi-Agent topology.
2) We implement a telemetry with circuit-breaking mechanisms integrated directly into the LangGraph orchestration logic based on MAST principles to identify and constrain catastrophic multi-agent failures.
3) We formalize the concepts of the *Multi-Agent Coordination Tax* and *Routing Regret*. Through an initial empirical evaluation on a customized 5-task pilot benchmark, we introduce a *Composite Reward* metric to demonstrate that Single-Agent Systems emerge as the dominant Oracle choice for these specific task typologies when latency and token efficiency are weighted alongside trajectory accuracy, establishing a foundational baseline for broader scaled evaluations.

II. Problem Statement

The fundamental problem addressed in this research is minimizing "Routing Regret" in agentic workflows. When an enterprise system defaults to a static Centralized MAS pipeline, it inevitably drains API tokens and wall-clock latency on sequential or tool-heavy tasks due to bidirectional state synchronization [4]. Consequently, the singular core problem is the dynamic, pre-execution assignment of an optimal system architecture for an incoming query, ensuring the router's prediction mathematically aligns with the most compute-efficient topology.

A. System Model and Composite Reward

To establish a theoretical ground truth (the "Oracle Baseline"), we evaluate each architecture's performance on a given query using a Composite Reward Score (R). This metric answers the important economic question: *Is the extra accuracy of a multi-agent system worth the latency and computational cost?*

While exact reward weighting varies according to specific enterprise Service Level Agreements (SLAs), recent infrastructure research heavily emphasizes that test-time scaling must be strictly bounded by latency and token constraints to remain viable in production [1], [2]. To model a representative enterprise deployment where latency is the primary concern, we establish a heuristic profile of: $\alpha = 0.50$ (Accuracy), $\beta = 0.40$ (Latency), and $\gamma = 0.10$ (Token Efficiency). To ensure fair task-level comparison, latency and token costs are min-max normalized against the worst-performing architecture ($\max_{a \in A}$) for that specific query:

$$R = \alpha \cdot \text{Accuracy} - \beta \cdot \frac{\text{Latency}}{\text{Latency}_{max}} - \gamma \cdot \frac{\text{Tokens}}{\text{Tokens}_{max}} \quad (1)$$

This specific distribution creates a strict zero-sum baseline: if an architecture achieves perfect trajectory accuracy (1.00) but serves as the maximum bound for both latency and token cost for that task, its net utility zeroes out ($0.50 - 0.40 - 0.10 = 0.000$). This ensures that the Oracle selection (O) identifies not just the most accurate system, but the one that provides the highest marginal utility per unit of time and compute:

$$O(q) = \arg \max_{a \in \{\text{SAS,CMAS}\}} R(a, q)$$

B. The Optimization Objective

The primary objective of the Dynamic Router is to minimize **Routing Regret**. Once the Oracle makes its choice, the regret is mathematically defined as the value the router left on the table by making a suboptimal prediction:

$$\text{Regret}(q) = R(O(q)) - R(\text{Router}_{selected}(q))$$

Suboptimal routing incurs significant latency and token penalties paid directly by the computing infrastructure. This is reflected as severe coordination loops.

III. Preliminary Idea, Methodology, and Analysis

To solve the routing regret problem, this research proposes the **Dynamic Architecture Router**. Rather than immediately generating a solution, our Router's main computational objective is reasoning for architectural assignment before workflow execution.

A. Methodology: System Architecture and Feature Extraction

The physical execution of this system relies on an advanced orchestration framework utilizing cyclical, stateful graphs via LangGraph. As identified by Nascimento et al. [15], autonomous self-adaptive frameworks are necessary to manage the unpredictable nature of multi-agent LLM systems.

Before a query enters the execution graph, the Dynamic Router acts as a zero-shot classification node via a Mistral-7B classifier. The Router analyzes the raw user prompt to extract three critical heuristic features, formulated as a feature vector $F(q) = [f_1, f_2, f_3]$:

1) **Sequential Depth (f_1):** The estimated length of the critical reasoning path.
2) **Parallelization Factor (f_2):** The degree to which the prompt contains orthogonal and separable subgoals.
3) **Tool Count Estimate (f_3):** the total predicted number of distinct API calls required.

TABLE I
PHASE IIA: INITIAL LLM EXECUTION RESULTS

Task ID	Category	SAS Acc.	SAS Lat.	SAS Tok.	CMAS Acc.	CMAS Lat.	CMAS Tok.
PCAB-Seq-01	Seq.	1.00	5.39s	3,849	0.67	10.51s	3,740
PCAB-Seq-02	Seq.	0.67	1.60s	1,756	1.00	22.91s	8,039
PCAB-Par-01	Par.	0.67	2.43s	2,386	1.00	**759.70s**	**11,427,875**
PCAB-Par-14	Par.	1.00	2.54s	1,728	1.00	7.24s	3,319
PCAB-Exp-03	Exp.	0.67	2.76s	2,367	1.00	**575.88s**	**4,905,968**

B. Initial Live Inference: Discovering the "Smoking Gun"

To establish a baseline for the Multi-Agent Coordination Tax, we initially executed live LLM evaluations across the Personalized Context Assembly Benchmark (PCAB), deployed locally on NVIDIA V100 GPUs using `Meta-Llama-3.1-8B-Instruct` via vLLM.

As documented in Table I, the initial CMAS topology achieved an impressive average accuracy of 0.93. However, this superficial success masked a catastrophic operational failure. Task `PCAB-Par-01` consumed over 11.4 million tokens and required nearly 13 minutes to execute. To mathematically quantify the severity of this breakdown, we evaluated these initial runs through our Composite Reward formula.

TABLE II
PHASE IIA: INITIAL COMPOSITE REWARD
$(\alpha = 0.50, \beta = 0.40, \gamma = 0.10)$

Task ID	SAS Reward	CMAS Reward	Oracle	Reward Gap
PCAB-Seq-01	**0.195**	-0.162	SAS	+0.357
PCAB-Seq-02	**0.285**	0.000	SAS	+0.285
PCAB-Par-01	**0.334**	0.000	SAS	+0.334
PCAB-Par-14	**0.308**	0.000	SAS	+0.308
PCAB-Exp-03	**0.333**	0.000	SAS	+0.333

As shown in Table II, despite perfect trajectory accuracy on multiple tasks, the catastrophic token bloat and execution delays of this initial CMAS pipeline triggered maximum latency and token penalties (-0.40 and -0.10, respectively). The composite reward function ruthlessly penalized this economic inefficiency, driving the CMAS reward down to a zero-sum floor of 0.000 or lower, and establishing the SAS as the dominant mathematical Oracle across 100% of the initial tasks.

C. MAST Telemetry and Multi-Layer Circuit Breaking

To accurately measure the true structural coordination tax and identify where the failure lies in the multi-agent loops, we stabilized the architecture. We implemented a comprehensive, 4-layer telemetry and circuit-breaking system derived from the Multi-Agent System Failure Taxonomy (MAST) [14]:

1) **Supervisor Level:** A computed *Loop Score Threshold* (≥ 0.5) and a strict *Turn Limit* (≤ 10) actively halt "System Design Issue: Infinite Loops." Furthermore, state history trackers detect $A \rightarrow B \rightarrow A \rightarrow B$ oscillations.
2) **Worker Level:** To prevent unconstrained token bloat, each sub-agent is wrapped in an exception handler with a strict recursion limit of 12. If a worker hits an context overflow, the system flags a "Worker Context Window Overflow," returns a partial result, and allows the supervisor to continue.
3) **SAS Level:** We implemented parameter hashing on tool signatures to log "Tool Explosions."
4) **Benchmark Level:** A catch-all trace logging mechanism for unhandled graphical propagation errors.

D. Circuit-Protected Inference: The Structural Tax

With the circuit breakers actively constraining infinite loops, we re-executed the benchmark sweep to observe the true baseline of the architectures.

TABLE III
PHASE IIB: CIRCUIT-PROTECTED LLM EXECUTION RESULTS

Task ID	Category	SAS Acc.	SAS Lat.	SAS Tok.	CMAS Acc.	CMAS Lat.	CMAS Tok.
PCAB-Seq-01	Seq.	1.00	5.45s	3,865	0.67	13.55s	4,100
PCAB-Seq-02	Seq.	0.67	1.58s	1,757	1.00	16.07s	5,617
PCAB-Par-01	Par.	0.67	2.37s	2,386	1.00	24.32s	7,247
PCAB-Par-14	Par.	1.00	2.32s	1,718	1.00	15.51s	5,698
PCAB-Exp-03	Exp.	0.67	2.71s	2,367	1.00	19.78s	6,747

As documented in Table III, the multi-layer circuit breakers were highly successful. Task `PCAB-Par-01`, which previously burned 11.4 million tokens, was cleanly executed in 24 seconds using only 7,247 tokens—a 99.94% reduction in waste.

However, the empirical telemetry traces the remaining inefficiencies directly to an exhaustive dispatch pattern, which is a form of a system design blind spot where the CMAS supervisor consistently struggles to confidently determine when it has enough data, prompting it to systematically dispatch every available worker and burn over 2800 tokens per unnecessary worker dispatch.

E. Protected Composite Reward Evaluation

To definitively prove the fundamental inefficiency of the CMAS topology, we evaluated the newly constrained data through our Composite Reward formula.

TABLE IV
PHASE IIB: PROTECTED COMPOSITE REWARD
$(\alpha = 0.50, \beta = 0.40, \gamma = 0.10)$

Task ID	SAS Reward	CMAS Reward	Oracle	Reward Gap
PCAB-Seq-01	**0.245**	-0.165	SAS	+0.410
PCAB-Seq-02	**0.265**	0.000	SAS	+0.265
PCAB-Par-01	**0.263**	0.000	SAS	+0.263
PCAB-Par-14	**0.410**	0.000	SAS	+0.410
PCAB-Exp-03	**0.245**	0.000	SAS	+0.245

As shown in Table IV and visually emphasized in Fig. 1, the SAS so far consistently outperforms CMAS with far superior reward. The bar chart illustrates a stark contrast, demonstrating that despite the circuit breakers reducing token waste by 99%, the CMAS consistently hits the 0.000 utility ceiling. This occurs because the "Exhaustive Dispatch" pattern ensures the CMAS always defines the maximum latency bound for the task.

Conversely, the SAS completes the same tasks with a much lower latency footprint, allowing it to capture the remaining

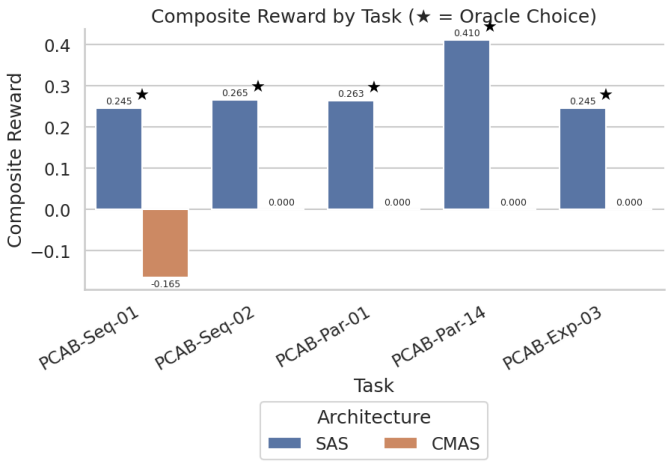

Fig. 1. Composite Reward by Task. The visual disparity clearly demonstrates how heavily CMAS is penalized for its structural inefficiencies. The Oracle choice (⋆) unanimously selects the SAS topology across all tasks.

0.25 to 0.41 points of utility. This gap represents the latency budget and efficiency of single-agent systems for these task scale. This data proves that even when multi-agent systems are "fixed" from a software perspective, their structural overhead makes them the inferior choice for the tested task typologies.

In parallel task scenarios such as PCAB-Par-01 and PCAB-Par-14, the CMAS architecture operates in its intended high-accuracy regime. However, the respective 0.263 and 0.410 reward gaps demonstrate that the speed of the SAS provides a superior overall efficiency. This suggests that the crossover point where CMAS becomes the mathematically optimal choice—exists only at a higher complexity threshold than currently represented in the pilot benchmark, further justifying the need for a Dynamic Router to identify these boundary conditions

F. Future Work and Scaled Evaluation

To definitively map the boundaries of the Multi-Agent Coordination Tax, the next and final phase of this research will scale the evaluation environment:

1) **WorkBench Dataset Transition:** Shifting from PCAB to WorkBench [16] to test architectures across 50 standardized, extended-horizon enterprise workflows.
2) **Ablation Studies (Feature Isolation):** Systematically isolating components of the Router's brain to validate the Mistral-7B LLM classification layer.
3) **Robustness & Adversarial Testing:** Injecting high-entropy noise into the simulation, including conflicting context and simulated API timeouts.
4) **Decentralized MAS (DMAS):** Introducing a third topology governed by peer-to-peer consensus rather than a centralized supervisor.

References

[1] Z. Wang, T. Zhang, H. Bai, L. Hou, X. Yu, W. Liu, S. Xiang, and L. Zhu, "Faster and Better LLMs via Latency-Aware Test-Time Scaling," in *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 2025, pp. 17 124–17 137.

[2] C. Snell *et al.*, "Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters," 2024, arXiv:2408.03314.

[3] J. Li, Q. Zhang, Y. Yu, Q. Fu, and D. Ye, "More Agents Is All You Need," *Transactions on Machine Learning Research*, 2024.

[4] Y. Kim, K. Gu, C. Park *et al.*, "Towards a Science of Scaling Agent Systems," 2025, arXiv:2512.08296.

[5] X. Li, "When Single-Agent with Skills Replace Multi-Agent Systems and When They Fail," 2026, arXiv:2601.04748.

[6] G. Park, S. Lee, and Y. Park, "Minimizing Response Latency in LLM-Based Agent Systems: A Comprehensive Survey," *IEEE Access*, vol. 14, pp. 1–20, 2026.

[7] N. Otoum and N. Elkhalili, "Methods and Techniques of Agentic Software Engineering: A Systematic Literature Review," *IEEE Access*, vol. 14, pp. 7443–7465, 2026.

[8] S. Chen *et al.*, "Sleipnir: Multi-agent based stock operator framework with interpretability," 2026, available on IEEE Xplore.

[9] H. Wu, Z. He *et al.*, "ChatEDA: A Large Language Model Powered Autonomous Agent for EDA," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 10, pp. 3184–3197, 2024.

[10] Y. Xia *et al.*, "LLM Experiments with Simulation: Large Language Model Multi-Agent System for Simulation Model Parametrization in Digital Twins," in *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2024.

[11] A. Al-Khatib *et al.*, "LLM-Driven Multi-Agent Architectures for Intelligent Self-Organizing Networks," *IEEE Network*, 2025.

[12] J. Liao *et al.*, "AutoForma: A Large Language Model-Based Multi-Agent for Computer-Automated Design," in *2024 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2024.

[13] L. Wu *et al.*, "A Large Language Model-Driven Multi-Agent Human-Machine Interaction Framework," in *2025 10th International Conference on Intelligent Informatics and Biomedical Sciences (ICIIBMS)*, 2025.

[14] Z. Hasan, A. Payani *et al.*, "Why Do Multi-Agent LLM Systems Fail?" 2025, arXiv:2503.13657, https://sites.google.com/berkeley.edu/mast/.

[15] N. Nascimento, P. Alencar, and D. Cowan, "Self-Adaptive Large Language Model (LLM)-Based Multiagent Systems," in *2023 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C)*, 2023, pp. 104–109.

[16] O. Styles *et al.*, "WorkBench: a Benchmark Dataset for Agents in a Realistic Workplace Setting," 2024, arXiv:2405.00823.